Java Programming: Object-Oriented Programming

Study notes for a document-based Retrieval-Augmented Generation (RAG) project

How to use these notes

These notes provide connected explanations, examples, terminology, and review questions. They are intended as a starter corpus for testing PDF extraction, chunking, embedding, and retrieval. Read the sections and ask questions whose answers appear in the material.

1. Object-oriented programming

Object-oriented programming (OOP) organizes software around objects that combine state and behavior. A class describes the structure and operations of objects. An object is an instance of a class. OOP can help model real-world concepts and separate responsibilities in a program.

2. Classes and objects

A class may contain fields, constructors, and methods. Fields represent state. Methods represent behavior. A constructor initializes a newly created object and has the same name as its class. The new keyword creates an object and invokes a constructor.

Code: class and object

```
class Student {
String name;
int mark;

Student(String name, int mark) {
this.name = name;
this.mark = mark;
}

void display() {
System.out.println(name + ": " + mark);
}
}

class Main {
public static void main(String[] args) {
Student s = new Student("Anu", 90);
s.display();
}
}
```

3. Encapsulation

Encapsulation keeps an object's internal state protected and exposes controlled operations. A common Java pattern is to make fields private and provide public methods for reading or changing them. Validation inside setter methods can prevent invalid state. Encapsulation reduces unintended dependencies on implementation details.

4. Inheritance

Inheritance lets a class extend another class and reuse or specialize its behavior. The extends keyword declares a subclass. A subclass inherits accessible members and may add new members or override inherited methods. Java classes have single class inheritance: a class can directly extend one class. Interfaces provide another way to express multiple types of capability.

Code: inheritance

```
class Animal {
void eat() {
```

```
System.out.println("Eating");
}
}

class Dog extends Animal {
void bark() {
System.out.println("Barking");
}
}
```

5. Polymorphism

Polymorphism means that one interface or parent type can refer to objects of different concrete types. With runtime polymorphism, an overridden instance method is selected according to the actual object at runtime. This allows code to work with a general type while each implementation supplies its own behavior.

6. Abstraction

Abstraction exposes essential operations while hiding implementation details. An abstract class can contain abstract methods and implemented methods. An interface defines a contract that implementing classes agree to support. Interfaces can also contain default and static methods.

7. Method overloading and overriding

Overloading uses the same method name with different parameter lists in one class or its inheritance hierarchy. Overriding supplies a new implementation of an inherited instance method using a compatible signature. The @Override annotation asks the compiler to verify that a method really overrides another method.

8. Constructors and this

A constructor initializes a new object. Constructors can be overloaded. The this reference points to the current object and is often used to distinguish fields from parameters. A constructor can delegate to another constructor in the same class using this(...).

9. Access modifiers

public members are accessible wherever the containing type is accessible. private members are accessible only within their declaring top-level class. protected members are accessible within the same package and in subclasses under Java's access rules. With no modifier, package-private access applies within the same package.

10. Composition versus inheritance

Composition builds a class using references to other objects that provide services. It often produces flexible designs because components can be replaced independently. Inheritance is useful for genuine subtype relationships, but deep inheritance hierarchies can make behavior difficult to understand.

11. OOP design example

A payment application might define a PaymentMethod interface with a pay(amount) operation. CardPayment and BankTransfer can implement that interface differently. A checkout service can depend on PaymentMethod rather than checking every concrete payment class. This is polymorphism combined with abstraction.

12. Review questions

1. What is the difference between a class and an object?

2. How does encapsulation protect data?

3. What keyword declares inheritance?

4. Explain runtime polymorphism.

5. How does an interface differ from an abstract class?

End of document — Java study corpus